ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет по компютърни системи и технологии (ФКСТ)

Специалност „Киберсигурност”

ПРОЕКТ

по дисциплина

„Глобални Бази Данни”

Тема:

„Онлайн система за изпити върху документно-ориентирана база данни (MongoDB)”

Изготвил: Ясен Василев Костов

Фак. №: 381223058

Ръководител: _____

София, 2026

СЪДЪРЖАНИЕ

1. Въведение
..... 3

2. Теоретична основа
..... 3

2.1. Документен модел и MongoDB 3

2.2. Сравнение с релационния модел 4

3. Реализация на системата
..... 5

3.1. Технологичен стек и архитектура 5

3.2. Модел на данните и валидация чрез $jsonSchema 6

3.3. Условни оператори и агрегационни конвейери 7

3.4. Потребителски интерфейс и потоци 8

4. Изводи и заключение
..... 9

5. Съкращения и литература
..... 10

1. Въведение

Управлението на учебния процес налага провеждането на голям брой контролни и изпитни мероприятия, като дистанционното и хибридното обучение допълнително повиши ролята на специализираните софтуерни платформи. Настоящият проект реализира онлайн система за изпити, построена изцяло върху документно-ориентираната база данни MongoDB. Целта е двояка – да се представят теоретично основните възможности на документния модел и да се покаже как те могат да бъдат приложени на практика.

В проекта умишлено се фокусираме върху три конкретни възможности на MongoDB: дефиниране на схемни ограничения чрез `$jsonSchema`, използване на условни оператори (`$cond`, `$switch`, `$ifNull`) и изграждане на агрегационни конвейери за статистически отчети. Уеб рамката е Django 4.2, а за връзка с базата се използва PyMongo – без Django ORM и без SQL база, за да остане фокусът върху нативните възможности на MongoDB.

2. Теоретична основа

2.1. Документен модел и MongoDB

NoSQL системите обединяват клас бази данни, които не следват класическия релационен модел. Разделят се на четири основни категории – ключ-стойност, колоночни, графови и документно-ориентирани. Общи техни характеристики са гъвкавостта на схемата, хоризонталната мащабируемост чрез sharding и предпочитанието към BASE пред ACID. Според теоремата на Брюър (CAP), разпределена система може едновременно да гарантира най-много две от трите свойства – консистентност, наличност и устойчивост на разделяне; MongoDB в класическата си конфигурация попада в категорията CP.

MongoDB е отворена документно-ориентирана база данни, разработена от MongoDB Inc. (първоначално 10gen) през 2007 г. Основната единица на съхранение е **документът** – JSON-подобен запис, вътрешно представен като BSON (Binary JSON). Документите се групират в **колекции**, аналог

на таблиците, но без фиксирана схема. Всеки документ получава уникален 12-байтов идентификатор `ObjectId`. MongoDB поддържа богат заявен език (`find`, `updateOne`, `$set`, `$inc` и др.) и мощен агрегационен конвейер – последователност от обработващи етапи, преминаващи през документите. За баланс между гъвкавост и строгост базата поддържа схемни валидатори, базирани на JSON Schema, които се задават на ниво колекция и отхвърлят документи, нарушаващи правилата.

2.2. Сравнение с релационния модел

Критерий	Релационен модел	Документен (MongoDB)
Структура	Таблици, редове	Колекции, BSON документи
Схема	Фиксирана, с миграции	Гъвкава, `$jsonSchema`
Връзки	Референции + JOIN	Вграждане или `$lookup`
Заявен език	SQL	Обектен + pipeline
Транзакции	ACID	Многодокументни ACID (v4.0+)
Мащабируемост	Вертикална	Хоризонтална (sharding)

Таблица 1: Релационен срещу документен модел

Документният модел е особено подходящ, когато данните са йерархични и схемата може да се променя динамично. В системата за изпити основните обекти – изпит с въпроси, опит с отговори – се представят естествено като документи с вложени масиви. Мощният агрегационен конвейер заменя нуждата от SQL при генериране на отчети, а схемните валидатори запазват част от строгостта на релационния подход, без да жертват гъвкавостта.

3. Реализация на системата

3.1. Технологичен стек и архитектура

Системата следва класически трислоен модел – браузър, Django приложение и MongoDB. Django изпълнява ролята на уеб рамка (шаблониране, сесии, URL-маршрутизация), а цялата работа с данни преминава през PyMongo директно, без Django ORM. Функционалните изисквания обхващат две роли:

- **Студент** – регистрация, вход, списък с налични и завършени изпити, полагане с таймер, автоматично предаване при изтичане на времето, преглед на резултата;
- **Администратор** – създаване и редакция на изпити и въпроси, активиране, преглед на обобщена статистика и специализирани отчети (преминали/непреминали, класация, трудност, почтеност).

Слой	Технология
Backend	Python 3.9, Django 4.2
База	MongoDB 7 + PyMongo 4
Frontend	Django templates + Bootstrap 5
JS	Vanilla JS (таймер, авто-предаване)
Пароли	Django PBKDF2 hashers

Таблица 2: Технологичен стек

3.2. Модел на данните и валидация чрез $jsonSchema

Моделът обхваща четири колекции: `users`, `exams`, `questions` и `attempts`. Връзките се реализират чрез `ObjectId` референции, като при нужда се извличат чрез `$lookup` агрегация.

Колекция	Ключови полета и ограничения
users	username (3–50 симв.), email (regex), role ∈ {student, admin}
exams	title (1–200), duration_minutes (1–480), passing_score (0–100)
questions	exam_id, text, options[2..6], correct_option, points (1–100)
attempts	user_id, exam_id, started_at, answers[], score, is_submitted

Таблица 3: Колекции и ограничения

Валидаторите се дефинират при създаване на колекцията и отхвърлят некоректни документи още от базата. Пример – част от валидатора на `users`:

```
db.create_collection("users", validator={
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["username", "email", "password_hash", "role",
"created_at"],
    "properties": {
      "username": {"bsonType": "string", "minLength": 3, "maxLength":
50},
      "email":    {"bsonType": "string",
                   "pattern": r"^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-
zA-Z]{2,}$"},
      "role":     {"enum": ["student", "admin"]},
      "created_at": {"bsonType": "date"},
    }
  }
})
```

Освен валидация, на `users.username` и `users.email` се налагат уникални индекси. За бърз достъп до опитите е създаден съставен индекс `attempts(user_id, exam_id)`.

3.3. Условни оператори и агрегационни конвейери

Агрегационният pipeline е централният инструмент за генериране на отчети. Примерно – функцията `get_pass_fail_rates()` класифицира всеки опит като „преминал” или „непреминал” спрямо прага на изпита чрез условен оператор `$cond`:

```
pipeline = [
  {"$match": {"is_submitted": True}},
```

```
  {"$lookup": {"from": "exams", "localField": "exam_id",
               "foreignField": "_id", "as": "exam_info"}},
  {"$unwind": "$exam_info"},
  {"$addFields": {
    "result": {"$cond": {
      "if":   {"$gte": ["$score", "$exam_info.passing_score"]},
      "then": "passed", "else": "failed"
    }}
  }},
  {"$group": {"_id": {"exam_id": "$exam_id", "result": "$result"},
              "count": {"$sum": 1}}},
]
```

Буквените оценки по NIST-подобна скала се получават с `$switch`, а докладът за почтеност използва комбинация от `$ifNull` (за защита от липсващо `finished_at`) и `$switch` за многостепенна класификация:

```
"integrity_status": {"$switch": {
  "branches": [
    {"case": {"$eq": ["$time_exceeded", True]},     "then": "Time
Exceeded"},
    {"case": {"$lt": ["$duration_seconds",
                      {"$multiply": ["$allowed_seconds", 0.1]}]}},
     "then": "Suspiciously Fast"},
  ],
  "default": "Normal"
}}
```

По време на разработката бе идентифицирана особеност на оператора `$round` – като израз в `$project` или `$addFields` той приема два аргумента (стойност и брой десетични знаци), но като акумулатор вътре в `$group` е унарен. Проблемът бе отстранен чрез разделяне на закръглянето в отделен `$addFields` етап след групирането.

3.4. Потребителски интерфейс и потоци

Интерфейсът е реализиран чрез Django шаблони и Bootstrap 5. Таблица 4 обобщава URL-картата на приложението.

URL	Роля	Предназначение
/register/, /login/	публично	Регистрация и вход
/dashboard/	студент	Налични и завършени изпити + статистика
/attempt/<id>/	студент	Полагане с таймер
/attempt/<id>/result/	студент	Резултат и верни отговори
/admin-panel/	администратор	Обобщение на изпитите
/admin-panel/exam/create/	администратор	Нов изпит с въпроси
/admin-panel/reports/	администратор	Четирите агрегационни отчета

Таблица 4: URL-карта на приложението

Поток при полагане на изпит. Студентът избира изпит и натиска „Start"; изгледът `start_exam` създава нов документ в `attempts` с начално време и празен масив от отговори. Отваря се страницата `/attempt/<id>/`, където JavaScript таймер брои оставащото време. При натискане на „Submit" или при изтичане на времето скриптът `exam.js` автоматично изпраща формата. Изгледът `submit_exam` изчислява процентния резултат, записва отговорите, вдига `is_submitted=true` и при просрочване – флага `time_exceeded`, който по-късно се използва в отчета за почтеност.

Поток при генериране на отчет. Изгледът `reports` извиква четирите агрегационни функции и подава резултатите към шаблона. Преди подаване полето `_id` се преименува на `status`, защото Django темплейтите забраняват достъп до атрибути, започващи с долна черта. Сесийното управление е стандартното Django сесийно cookie, а паролите се хешират и проверяват с `django.contrib.auth.hashers`.

4. Изводи и заключение

Проектът реализира работеща уеб-базирана система за онлайн изпити, построена изцяло върху MongoDB. Подходът – Django като уеб рамка без ORM, в комбинация с PyMongo – позволи да се демонстрират нативните възможности на документно-ориентираната база данни без допълнителни абстракции.

Валидирането чрез `$jsonSchema` показа, че MongoDB може сама да гарантира целостта на данните – всички ограничения (задължителни полета, типове, диапазони, регулярни изрази, минимална и максимална дължина на масиви) се прилагат прозрачно за приложението. Условните оператори `$cond`, `$switch` и `$ifNull` позволиха бизнес логика (буквени оценки, статус „преминал/непреминал", класификация на почтеността) да се изнесе директно в базата, което намалява обема на Python кода и обема на пренасяните данни. Агрегационните конвейери демонстрираха, че сложни статистически отчети – средни, минимални, максимални стойности, проценти, класации – се изграждат с няколко етапа (`$match`, `$group`, `$lookup`, `$unwind`, `$project`), без да е нужен SQL.

Получените знания са директно приложими в по-мащабни проекти – системи за електронно обучение, управление на съдържание и аналитика в реално време – където документно-ориентираните бази данни са често срещан избор. В заключение може да се каже, че MongoDB, подкрепен от добре дефинирани схемни валидатори и агрегационни конвейери, предоставя достатъчен набор от инструменти за изграждане на пълноценно уеб приложение от промишлен клас.

5. Съкращения и литература

Съкращения. **ACID** – Atomicity, Consistency, Isolation, Durability; **API** – Application Programming Interface; **BASE** – Basically Available, Soft state, Eventual consistency; **BSON** – Binary JSON; **CAP** – Consistency, Availability, Partition tolerance; **CRUD** – Create, Read, Update, Delete; **HTTP** – HyperText Transfer Protocol; **JSON** – JavaScript Object Notation; **NoSQL** – Not Only SQL; **ORM** – Object-Relational Mapping; **PBKDF2** – Password-Based Key Derivation Function 2; **SQL** – Structured Query Language; **URL** – Uniform Resource Locator.

Използвана литература:

1. Chodorow, K., „MongoDB: The Definitive Guide”, 3rd Edition, O'Reilly Media, 2019.
2. Banker, K., Garrett, D., Bakkum, P., Verch, S., „MongoDB in Action”, 2nd Edition, Manning Publications, 2016.
3. Sadalage, P., Fowler, M., „NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence”, Addison-Wesley, 2013.
4. Brewer, E., „Towards Robust Distributed Systems”, PODC Keynote, 2000.
5. Gilbert, S., Lynch, N., „Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, ACM SIGACT News, vol. 33, no. 2, 2002.
6. MongoDB Inc., „MongoDB Manual”, Official Documentation, https://www.mongodb.com/docs/manual/, 2025.
7. MongoDB Inc., „Aggregation Pipeline Reference”, https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/
8. MongoDB Inc., „Schema Validation with $jsonSchema”, https://www.mongodb.com/docs/manual/core/schema-validation/
9. Django Software Foundation, „Django Documentation”, version 4.2, https://docs.djangoproject.com/en/4.2/
10. MongoDB Inc., „PyMongo – Python Driver for MongoDB”, https://pymongo.readthedocs.io/
11. JSON Schema, „JSON Schema Specification Draft 2020-12”, https://json-schema.org/specification.html
12. Codd, E. F., „A Relational Model of Data for Large Shared Data Banks”, Communications of the ACM, vol. 13, no. 6, 1970.